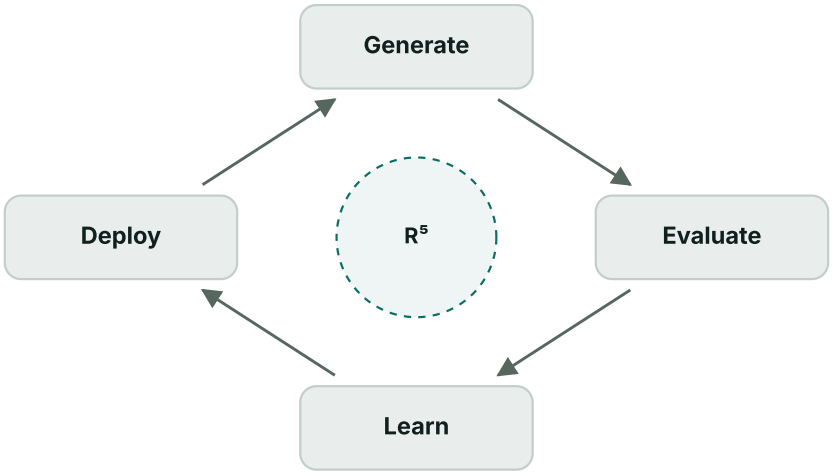
Building Agents That Learn

1. The Self-Improvement Flywheel

- **Generate** a solution → **Evaluate** it against quality criteria → **Learn** (update or tune) → **Deploy** the improvement → repeat
- **Evaluate** is the crucial stage: a system can only improve what it can measure
- **Learn** must balance **exploitation** (refining known-good strategies) and **exploration** (trying new ones), or it stagnates

> ⚠ Learning from bad data or a hallucinating evaluator turns the flywheel into a vicious cycle of degradation.

✎ DIAGRAM



The flywheel is the what; R^5 is the how that keeps it safe.

2. The R^5 model

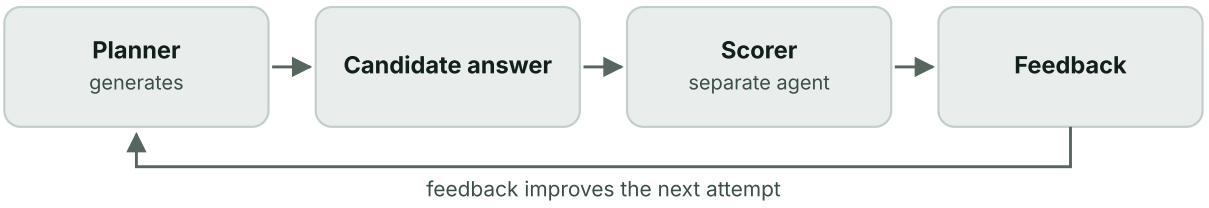
- **Relax**: actively manage context and latency so agents stay coherent under load
- **Reflect**: deliberate self-critique checkpoints, improving mid-run without retraining
- **Reference**: provenance (citations, retrieval traces) so outputs are attributable
- **Retry**: adaptive, reasoned error handling, not blind repetition
- **Report**: quantify factuality, consistency and process quality to close the loop

3. Patterns for learning safely

- **Planner + Scorer (hybrid) architecture**: separate the generator from the evaluator, because an agent judging itself rationalises its errors. The foundational pattern

- **Custom evaluation metrics**: task-specific scores, e.g. the quality of each step, not just the final answer
- **Preference-controlled synthetic data**: training data steered towards desired preferences
- **Tuning**: SFT and PEFT for capability, then **DPO** (learn from better-vs-worse pairs) for alignment
- **Coevolved training**: generator vs critic, both improving · **Red teaming**: attack your own agents first

✎ DIAGRAM



Never let one agent both create and grade its own work.

4. Tokenomics and ROI

- Multi-agent systems fan out into many calls that each resend context, so **cost scales with agents × steps**, not just tasks. Levers: route easy sub-tasks to cheaper models, trim and cache context, cap output and reflection loops, batch async work, track cost per task
- **ROI** = tasks × time saved × loaded rate × **realistic automation rate** − run and ops cost
- The honest variable is the automation rate: demos automate 100% of the happy path; production automates **60 to 80%** and escalates the rest

SAY IT

"You can't let one agent both create and judge its own work, so I'd split the generator from the scorer and use R^5 to keep the learning loop safe."